
Functors

We now change direction a little and address the concept of morphisms between categories. It is the “sensible” definition, using the principle of preserving structure.

20.1 Making up the definition

In Section 13.5 we briefly introduced the idea of a functor. The idea is that functors are the structure-preserving maps between categories. If you can’t remember what the definition was at this point, I hope you might feel you could start making it up for yourself using the idea of preserving structure. I think this is the best way to get math into your brain: understand the principles so that you can create it for yourself. This is why I think that structural math shouldn’t involve memorization, though some people do a bait-and-switch and claim that by memorization they mean “put into your memory”. I think that’s not what memorization typically means, and that we should keep separate the idea of rote memorization (without meaning) and the sort of process I’m calling structural, where you deeply understand a structural principle so that it becomes part of your consciousness.

The idea of a functor is that it’s a function on underlying data, preserving the structure. In fact, now that we have expressed the definition of category in two slightly different ways I will also express the definition of functor in two slightly different ways.

Definition by homsets

The first definition we saw of (small) category has the underlying data as

- a set of objects
- for every pair of objects a set of morphisms, called a homset.

In this case the definition of functor consists of a function on sets of objects, and a function on every homset.

Definition 20.1 Let $\mathcal{C}$ and $\mathcal{D}$ be (small) categories. A *functor* $F: \mathcal{C} \rightarrow \mathcal{D}$ is given by

- a function $F: \text{ob } \mathcal{C} \rightarrow \text{ob } \mathcal{D}$, and
- for all objects $x, y \in \mathcal{C}$, a function $F: \mathcal{C}(x, y) \rightarrow \mathcal{D}(Fx, Fy)$

satisfying the following conditions called *functoriality*:

- on identities: for all objects $x \in \mathcal{C}$, $F(1_x) = 1_{Fx}$ and
- on composites: for all morphisms $x \xrightarrow{f} y \xrightarrow{g} z \in \mathcal{C}$, $F(g \circ f) = Fg \circ Ff$.

However, it is often stated in the “more general” way below, which does not specifically refer to functions, so that the definition works for categories of all sizes (whereas functions can only be invoked on *sets* of things). It is also a little more like how we typically use functors.

You might be perplexed by the way the word “associates” is used in this more general definition. It’s not really how we use it in normal English, but is how we use it in math. You might want to start by reading the definition below, and then read the footnote[†] for further unraveling.

More general definition of functor

Let $\mathcal{C}$ and $\mathcal{D}$ be categories. A *functor* $F: \mathcal{C} \rightarrow \mathcal{D}$ associates:

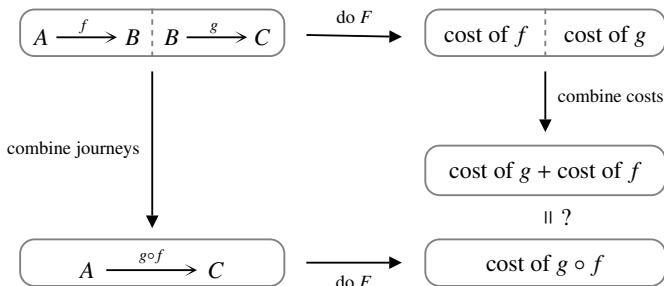
- to every object $x \in \mathcal{C}$ an object $Fx \in \mathcal{D}$,
- to every morphism $x \xrightarrow{f} y \in \mathcal{C}$ a morphism $Fx \xrightarrow{Ff} Fy \in \mathcal{D}$

satisfying functoriality as above.

We could think of functoriality in terms of plane ticket prices in the following way. Suppose we have one category consisting of journeys by plane, so a morphism from A to B is a route from A to B by plane (possibly with layovers). Composition is then just doing one journey followed by another. We also have a category of numbers, and we could try and map journeys to numbers by their prices as an attempt at a functor F . The question of functoriality is then this: suppose you’re flying from A to B to C . If you buy the ticket from A to B and

[†] A more natural phrasing in normal English would be “A functor F associates an object Fx to every object x ” but in math we turn it round so that the object that comes first in the logic also comes first in the sentence. The other slight oddity is that we say “associates to” rather than “associates with”. Here “associate” is an active verb, perhaps more like “assign”. What this definition really says is “Given any object $x \in \mathcal{C}$, the functor F produces an object $Fx \in \mathcal{D}$ ”, and similarly for morphisms.

then the ticket from B to C as two separate journeys, does it cost the same as buying the whole ticket from A to C as one journey? The following is a schematic diagram depicting this situation, which means the triangular-headed arrows aren't morphisms in any particular category, but rather, they are processes.



There are two entries for the bottom right corner because *a priori* we don't know if they're the same. One comes from going around the top-and-right edges of the square, and the other comes from going around the left-and-bottom. I would say the answer is that they are not in general the same. Airline ticket pricing is somewhat obscure (especially for international routes), and I would typically not expect anything as simple as adding the prices together.

Definition by graphs

We saw another definition of category, where the underlying data is a diagram of sets and functions as shown on the right.

$$C_1 \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} C_0$$

Things To Think About

T 20.1 What do you think the underlying data for a functor should be in this context?

In this context, the underlying data for a functor is a pair of functions F_1 and F_0 as shown on the right, making the diagram “serially commute”.

$$\begin{array}{ccc}
 C_1 & \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} & C_0 \\
 F_1 \downarrow & & \downarrow F_0 \\
 D_1 & \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} & D_0
 \end{array}$$

This is something we sometimes say about diagrams involving parallel arrows. If a diagram “serially” commutes it means that only the sub-diagrams[†] involving corresponding arrows commute. In the diagram above there is a square

[†] By “sub-diagram” I mean a part of the diagram in question.

involving just the top arrow s of each parallel pair (together with the vertical arrows), and there is a square involving just the bottom arrow t of each parallel pair. Each of those two squares is required to commute, but nothing mixing up an s and a t is required to commute.

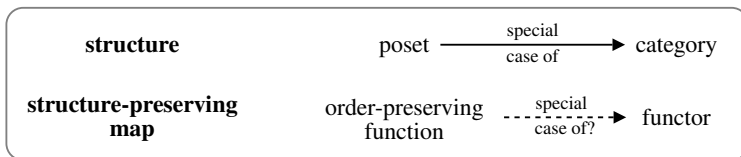
The definition of category and functor by graphs is of interest for abstract reasons, as we'll see briefly in the last chapter, Chapter 24 on higher dimensions. However, the full definitions are a little beyond our scope.

20.2 Functors between small examples

We'll start by looking at our usual examples of mathematical structures expressed as categories.

Posets as categories

Recall that a poset is a category in which there is at most one morphism between any two objects. Now, since posets are a special case of categories, we might sensibly wonder: if we regard posets as categories, do functors between them correspond to order-preserving functions? Here is a schematic diagram:



The dotted arrow could mean different things: it could mean that we need to take special cases of functors between posets (expressed as categories) or it could be that we take all the functors, and it's only a special case because we've already restricted the categories. The latter turns out to be true. Later on we'll see that the above "schematic" diagram can be turned into a rigorous piece of theory in its own right.

Things To Think About

T 20.2 Consider posets A and B ordered by $\leq$. Express them as categories, and show that a functor $F: A \rightarrow B$ is precisely an order-preserving function. In what multiple ways have I abused notation here? Does it bother you?

We express a poset as a category by having a morphism $a \rightarrow a'$ whenever $a \leq a'$. A functor $F: A \rightarrow B$ in that case gives us:

- on objects: a function on objects, that is, a function on the underlying sets $A \rightarrow B$,
- on morphisms: for every $a \rightarrow a'$ we get a morphism $Fa \rightarrow Fa'$, that is, whenever $a \leq a'$ we have $Fa \leq Fa'$, which is exactly telling us we have an order-preserving function.

I have abused notation by using the same notation for the underlying sets, the posets, and also the posets expressed as categories. Personally I think that this is unambiguous and, in this case, clearer than the notational mess that would result if we tried to use different notation for those three structures. Furthermore, I am trying to encourage the idea of thinking of those three things simultaneously, not separately.

Monoids and groups as categories

Recall that a monoid is a category with only one object. Unraveling that, we see it is a set with a binary operation that is associative and unital. A group is then a category with only one object in which every morphism has an inverse, though that's more of a characterization than a direct definition.

Things To Think About

T 20.3 What could we then ask ourselves about monoid/group homomorphisms, analogously to what we asked for posets? Can you answer it?

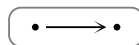
The analogous question is: as monoids are a special case of categories, are monoid homomorphisms precisely the functors between those categories? The answer is yes. Functoriality requires preserving identities and preserving composition. Those precisely correspond to the two conditions for a monoid homomorphism: preserving identities and preserving the binary operation.

20.3 Functors from small drawable categories

Functors *out of* a small drawable category are a way of abstractly finding structure in another category. This is similar to the fact that a function from any 1-element set to a set X simply picks out an element of X .

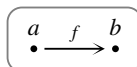
Things To Think About

T 20.4 Here is the “quintessential” category consisting of one arrow, as we mentioned in Section 11.1.

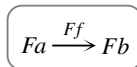


What is a functor from this to **Set**? What about from this to the category of monoids **Mon**? From this to any category $\mathcal{C}$?

Let's call this category $\mathbb{A}$ (for “arrow”), and name its objects and morphism as shown here.



To specify a functor $F: \mathbb{A} \rightarrow \mathbf{Set}$ we just have to specify sets and functions as shown on the right.



There is nothing left to specify or check as we know that identities must go to identities, and there are no non-trivial composable morphisms anyway. Note that Fa and Fb could be the same set, and Ff could itself be the identity: identities *must* be mapped to identities, but non-identities are allowed to be mapped to identities. So a functor out of the category $\mathbb{A}$ just picks out a function (and by implication, source and target sets as well).

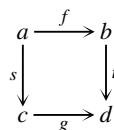
Similarly, a functor $\mathbb{A} \rightarrow \mathbf{Mon}$ just picks out a monoid homomorphism (and by implication, a pair of monoids for the source and target as well). In general a functor $\mathbb{A} \rightarrow \mathcal{C}$ just picks out a morphism in $\mathcal{C}$.

In category theory we are liable to say “a functor $\mathbb{A} \rightarrow \mathcal{C}$ is a morphism in $\mathcal{C}$ ” with a rather categorical[†] use of the word “is”. When we look at natural transformations we’ll see a way to organize both functors and morphisms into categories themselves, and then say something more precise about “is”.

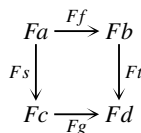
Things To Think About

T 20.5 Can you show that a functor must map a commutative square to a commutative square? Can you thus make a category $\mathbb{J}$ such that a functor from $\mathbb{J} \rightarrow \mathcal{C}$ just picks out a commutative square in $\mathcal{C}$?

Suppose we have a commutative square as shown here in some category $\mathcal{E}$, and a functor $F: \mathcal{E} \rightarrow \mathcal{C}$.



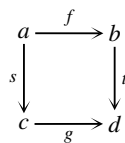
Then we certainly get a square in $\mathcal{C}$ as shown here, and the question is whether or not it commutes, that is, whether the composite around the top-and-right equals the composite around the left-and-bottom.



Now, the top-and-right composite is $Ft \circ Ff$ which equals $F(t \circ f)$ by functoriality of F ; similarly the left-and-bottom composite equals $F(g \circ s)$. The commutativity of the original square tells us $t \circ f = g \circ s$, so we must also have $F(t \circ f) = F(g \circ s)$. So the second square commutes.

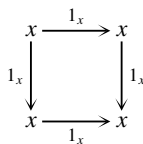
[†] As always, by “categorical” I mean “pertaining to category theory” rather than “decisive and clear”.

This means that we can usefully take a category $\mathbb{J}$ to be a “quintessential commutative square category”, that is, it consists of a commutative square as shown on the right, and nothing else.



Then a functor $\mathbb{J} \rightarrow \mathcal{C}$ will simply pick out a commutative square in $\mathcal{C}$. As with the “quintessential morphism”, the commutative square in $\mathcal{C}$ need not have all its corners actually being different objects.

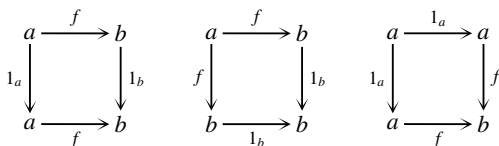
In fact, the functor could perfectly well pick out a commutative square as shown here, which we might call *degenerate*; it’s really a single point.



Things To Think About

T 20.6 Can you think of some ways that a functor $\mathbb{J} \rightarrow \mathcal{C}$ could pick out a single morphism in $\mathcal{C}$ expressed as a degenerate commutative square? That is, some of the edges of the square will be identities.

Here are some ways we could pick out a single morphism $a \xrightarrow{f} b$ as a degenerate square.



This might seem a bit of a futile exercise but this sort of “degeneracy” can be useful, especially when building up an entire situation out of triangles, which often happens in topology. Allowing degenerate ones gives us important flexibility and generality in the formalism.

We have used a “quintessential” category[†] of a certain shape to pick out diagrams of that shape in another category $\mathcal{C}$. We can now take this idea and use it as an abstract definition of diagrams in a category.

Definition 20.2 Let $\mathbb{J}$ be a finite category and $\mathcal{C}$ be any category. Then a *diagram of shape $\mathbb{J}$ in $\mathcal{C}$* is a functor $\mathbb{J} \rightarrow \mathcal{C}$.

This is the abstract formalism we use to define general universal properties, as we briefly alluded to in the previous chapter. To define a general limit we start with a diagram of shape $\mathbb{J}$ expressed as a functor. We then define cones over

[†] The sorts of categories I’m calling “quintessential” are sometimes called “walking” (as coined by Baez and Dolan), like a man with a mustache that is so big it looks like the man exists only to support the mustache: he’s a walking mustache.

that diagram, which we can also do abstractly, but not until we've seen natural transformations.

Another “quintessential” category we could look at is a quintessential isomorphism. In order to use such a category to pick out an isomorphism in another category, we need to know that if we apply a functor to an isomorphism it will still be an isomorphism in the target category.

Things To Think About

T 20.7 See if you can show that the above fact is true about functors and isomorphisms, and construct a “quintessential isomorphism” category such that a functor out of it just picks out two objects and an isomorphism between them.

First it's good practice for us to state this precisely.

Proposition 20.3 *Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor, and f an isomorphism in $\mathcal{C}$ with inverse g . Then Ff is an isomorphism in $\mathcal{D}$ with inverse Fg .*

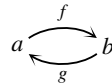
Proof To prove that Ff and Fg are inverses we compose them and check that the composite is the identity both ways round. Now

$$\begin{aligned} Fg \circ Ff &= F(g \circ f) && \text{by functoriality} \\ &= F1 && \text{since } f \text{ and } g \text{ are inverses} \\ &= 1 && \text{by functoriality} \end{aligned}$$

and similarly (dually) for $Ff \circ Fg$. Thus Fg is an inverse for Ff and Ff is an isomorphism as claimed. $\square$

Note that I wrote 1 without being very specific what object this is the identity on. It doesn't really matter: it's the identity on the object it needs to be the identity on.

We can make a “quintessential isomorphism” category as shown here, in which f and g are inverses.



A functor from this category to any category $\mathcal{C}$ will then pick out two objects in $\mathcal{C}$ and a pair of inverse morphisms between them. As usual, these two objects could be the same, and the inverse morphisms could both be the identity.

The fact that an isomorphism is still an isomorphism after applying a functor is called *preservation* and we'll come back to that shortly, after we've looked at some more examples of functors.

20.4 Free and forgetful functors

We have secretly been using various functors to **Set** when thinking about sets with structure. For example when we were looking at products of posets we took products of their underlying sets and then constructed a “natural” ordering on that set. Without saying so, we were invoking a functor $\boxed{\mathbf{Poset} \rightarrow \mathbf{Set}}$, which takes each poset and “forgets” that it’s a poset. That is, the functor sends each poset to its underlying set. On morphisms the functor sends an order-preserving function to its underlying function. We did something similar for products of groups, starting from the product of their underlying sets and constructing a binary operation for it.

Functors that forget structure are called forgetful functors. They are often denoted U (for “underlying”, perhaps) and this shows us that when we’re talking about a poset A and its underlying set, we should technically call its underlying set UA . Another way we could avoid abusing notation is to call the poset $(A, \leq)$ and then define $U(A, \leq) = A$. However, I think both of these systems are often a little tedious and not terribly enlightening.

Things To Think About

T 20.8 What other forgetful functors have we implicitly thought about?

Here are some other forgetful functors we have met involving sets-with-structure, although we didn’t formally say it that way at the time.

$$\begin{array}{l} \mathbf{Mnd} \rightarrow \mathbf{Set} \\ \mathbf{Grp} \rightarrow \mathbf{Set} \\ \mathbf{Top} \rightarrow \mathbf{Set} \end{array}$$

Forgetful functors can also just forget *some* of the structure.

For example, a group is a monoid with some extra structure (inverses) so we have this forgetful functor:

$$\mathbf{Grp} \rightarrow \mathbf{Mnd}$$

We could also take an Abelian group and forget that it’s Abelian, giving this forgetful functor:

$$\mathbf{Ab} \rightarrow \mathbf{Grp}$$

Similarly we could take a totally ordered set and forget that the ordering was total, giving this forgetful functor:

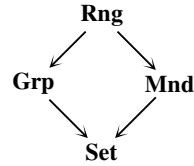
$$\mathbf{Tst} \rightarrow \mathbf{Pst}$$

Sometimes the structure on a set is actually a combination of two types of structure. For example, a *ring* is a set with two binary operations that interact in a particular way. The two binary operations are often thought of as addition and multiplication, and the integers are a key example: we can add and multiply integers, and moreover, addition and multiplication interact nicely via a distributive law $\boxed{a \times (b + c) = (a \times b) + (a \times c)}$.

We express this abstractly by saying we have a group with respect to one

binary operation and a monoid with respect to the other, and the two binary operations have to interact according to the distributive law as above.[†]

I hope you can guess what ring homomorphisms are — they are simultaneously group homomorphisms and monoid homomorphisms. This makes a category **Rng** of rings and their homomorphisms, and we have a system of forgetful functors like this.



Things To Think About

T 20.9 Do you think this is a pullback square?[‡]

Forgetful functors might seem a bit contentless and it's true that we often use them without thinking about them very explicitly, as indeed we did in some earlier chapters of this book. However, one very powerful aspect of them is the relationship they often have with a functor going the other way which does the “opposite” of forgetting structure: it creates structure freely and is called a *free* functor. By contrast, sometimes forgetful functors do not have a relationship with such a functor, and that is in itself interesting.

Another kind of functor we've implicitly seen that doesn't exactly do much is a “special case” functor. Posets are a special case of categories: we can express any poset as a category with at most one morphism between any two objects. Order-preserving functions between the posets then become functors between those special categories. Thus we get a functor **Pst** $\rightarrow$ **Cat**.

Likewise we can express monoids as one-object categories. If we decide in advance what the single object is going to be (say, it's always going to be the symbol $*$) then we get a functor **Mnd** $\rightarrow$ **Cat**.

Free functors are much more difficult than “special case” functors or forgetful functors, but we can try them for monoids, where they're not too bad. The idea is to make a functor **Set** $\rightarrow$ **Mnd** that takes a set and turns it into a monoid “freely”. This means: without imposing any unnecessary constraints or using any extra information. We have to do exactly what is required to make it a monoid, and do it without imposing any of our own choices. A monoid is a set with some extra structure, so we need to create that structure from somewhere. The extra structure we need is an identity and a binary operation.

We already saw something like this free construction in Section 11.2 when we looked at a category starting from a single object with a single arrow f from the object to itself. We said that if we did not impose any extra conditions

[†] We also need to specify the version for $(b + c) \times a$ if multiplication is not commutative.

[‡] It isn't, because a ring isn't just a group and a monoid with the same underlying set: those structures have to satisfy a distributive law. That's not a proof, but it's the main idea.

on the composition in this category then, in addition to f , the category would have an identity 1 , and composites f^n for every $n \in \mathbb{N}$, produced by composing the arrow with itself any finite number of times. In fact this is the free monoid generated by a single element f .

Things To Think About

T 20.10 Suppose we start with two elements a, b . You can think of them as arrows in a category with one object, or just elements that we're going to combine using a binary operation. If we keep composing freely without any extra conditions, what composites will we get?

We are starting with a set $\{a, b\}$ and we want to make it into a monoid. What could be the result of doing the binary operation? We need results for doing the binary operation on pairs of elements as shown on the right.

$a \circ a$
$a \circ b$
$b \circ a$
$b \circ b$

However, the answers can't be a or b because that would impose some unnecessary constraints on the monoid. That is, it would impose some equations that aren't demanded by the definition of a monoid.

The only thing we can do to avoid that is to declare "it is what it is" or something like that: the answer to $a \circ a$ is just that, a new element $a \circ a$ whose entire purpose in life is to be the answer to $a \circ a$. We do something similar for the other answers. Typically we get bored of writing the symbol $\circ$ so we just write these new elements as $\boxed{aa, ab, ba, bb}$.

Now, we can't stop there: we've added in these new elements, so now we need the answers to binary operations involving these new elements. So we need things like $\boxed{a(ba), b(ba), b(ab), \dots}$

We get to save a little effort because associativity tells us it doesn't matter where those parentheses go, but we still can't stop because every time we make new elements we will then have to produce answers to binary operations with them. It's just like producing all f^n from a single element f except now we are building up from two elements instead of one. These are called *words* in the elements a and b , because they really are quite a lot like words using an alphabet, it's just that in our example our alphabet only has two letters. A word is a finite string of letters, and the letters are allowed to repeat themselves.

The binary operation is called *concatenation*, and consists of just sticking two words together to make a longer word, as is often done in German to great effect to produce wonderful words like "Schadenfreude" and "Liebestod". (English has some German roots and we do sometimes do this in English too, with words like handbag and toothpaste.)

We haven't yet dealt with the identity: that's the empty word, a word with

no letters. It's not a space, because a space is an actual character, and we need something that really has nothing in it so that when we concatenate it with another word nothing happens to that word.

So we can start with any set A and produce the monoid of (finite) words in the elements of A . This is called the *free monoid on A* .

Things To Think About

T 20.11 Given sets A and B and a function $f: A \rightarrow B$ can you produce a monoid homomorphism from the free monoid on A to the free monoid on B ?

Let us write FA for the free monoid on A . Given a function $f: A \rightarrow B$ we can then make a monoid homomorphism $Ff: FA \rightarrow FB$ as follows. An element of FA is a word $a_1a_2 \cdots a_n$ where each $a_i \in A$ (and it is possible for the a_i to be equal for different i). We then define

$$Ff(a_1a_2 \cdots a_n) = fa_1fa_2 \cdots fa_n$$

which is a word in the elements of B . This is a bit like making a very basic code where you replace every letter of the alphabet by another one, say

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Q	W	E	R	T	Y	U	I	O	P	A	S	D	F	G	H	J	K	L	Z	X	C	V	B	N	M

and then any normal word can be turned into a coded word such as

$$\text{DELICIOUS} \longmapsto \text{RTSOEOGXL}$$

For codes like this it's important that the function on letters is an isomorphism otherwise it won't be decodable, but for our free monoids it needn't be an isomorphism; for example, if we used the function $A \rightarrow \{*\}$ then every word in the elements of A would just become a string of $*$'s, like when you type in a password but it stays hidden on the screen. We can check that all this makes F into a functor $\mathbf{Set} \rightarrow \mathbf{Mnd}$. It is called the *free monoid functor*.

The free monoid on a single element is isomorphic to the natural numbers (under addition). This is an abstract encapsulation of the fact that we basically make the natural numbers by starting with the number 1 and adding it together repeatedly. In order for this really to be the free monoid we must include 0 as the additive identity, and this is one reason that I typically prefer including 0 in the natural numbers. Being a "free" structure is a type of universal property, so if we include 0 then the natural numbers have a good universal property among monoids and thus among categories. Whereas if we don't include 0 then the natural numbers only have a good universal property among sets with a binary operation but not necessarily an identity; the lack of identity means that such structures are not an example of a category. (However they are studied in their own right and are called *semi-groups*.)

We can do a similar free construction for groups, but it's more complicated because we have to add in inverses as well. Essentially we throw in new elements to be inverses, and then make words out of elements and their inverses, but we have to be careful to make sure that if a letter finds itself next to its inverse in the word then they cancel out. The free group on a single element happens to be Abelian and in fact is isomorphic to the integers. This encapsulates the fact that the integers are made just from the number 1, as with the natural numbers, except now we want to be able to subtract as well as add.

In earlier chapters we were implicitly using forgetful functors to help us with universal constructions. The idea is that if functors interact well with the structure in their source and target categories, they can help us understand the structure in one via the structure in another. This good interaction can happen both forwards and backwards, and this is the idea of “preserving” and “reflecting” structure.

20.5 Preserving and reflecting structure

When we have a functor, say $\mathcal{C} \xrightarrow{F} \mathcal{D}$, one of the things we do is look at how this relates structure in $\mathcal{C}$ to structure in $\mathcal{D}$, both forwards and backwards. That structure might be any of the things we've seen so far such as isomorphisms, commutative squares, monics and epics, terminal and initial objects, products and coproducts, pullbacks and pushouts, or any limits and colimits.

In the forwards direction we have the question of *preservation*: given a type of structure in $\mathcal{C}$, if we apply F does it still have that structure in $\mathcal{D}$?

In the backwards direction we have the question of *reflection*: if we apply F to something in $\mathcal{C}$ and the result has a particular structure in $\mathcal{D}$, can we conclude that it already had that structure in $\mathcal{C}$?

So far we have seen that isomorphisms and commutative diagrams are preserved by all functors. However, they are not necessarily reflected.

Things To Think About

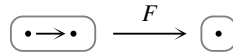
T 20.12 Can you construct some very small examples of categories and a functor $\mathcal{C} \xrightarrow{F} \mathcal{D}$ in which:

1. There is a morphism $f \in \mathcal{C}$ that is not an isomorphism although Ff is an isomorphism in $\mathcal{D}$.
2. There is a non-commutative diagram in $\mathcal{C}$ such that when we apply F we get a commutative diagram in $\mathcal{D}$.

Hint: for both situations you could take $\mathcal{D}$ to be the category with one object and one (identity) morphism.

In each case we can take $\mathbb{C}$ to be a “quintessential” or “walking” category for the structure in question, and we take $\mathbb{D}$ to be the category $\mathbb{1}$ with one object and one (identity) morphism.

Consider first the “quintessential morphism” category and a functor to $\mathbb{1}$ as shown here.



As the target category has only one object and only one morphism, we have no choice about where F can send everything: every object has to go to the single object and every morphism goes to the identity. Functoriality has to hold because the only morphism in the target category is the identity, so every equation of morphisms just reduces to an identity equalling itself.

Now we just check that this functor sends a non-isomorphism to an isomorphism. The single non-trivial morphism in $\mathbb{C}$ is not an isomorphism as it has no inverse, but it is mapped to the identity in $\mathbb{D}$ which is an isomorphism. So we see that F does not reflect isomorphisms.

We can do something similar with a “quintessential non-commutative diagram” category. We could take a square that does not commute.

In fact it could just be a triangle that doesn’t commute, or indeed a pair of parallel arrows.



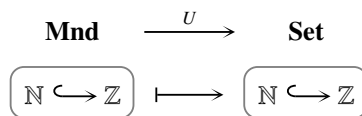
Technically if we have two parallel arrows that are not equal then the diagram does not commute, as there are two paths with the same endpoints whose composites are not the same. (In fact that is true if we just take a non-trivial loop, that is, a non-identity arrow from an object to itself.) Now if we again take the functor to the category $\mathbb{1}$, the parallel arrows will both be mapped to the identity, so the non-commutative diagram is mapped to a commutative diagram. This is thus a functor that does not reflect commutative diagrams.

Things To Think About

T 20.13 In Section 15.6 we saw that the monoid morphism $\mathbb{N} \rightarrow \mathbb{Z}$ sending everything to itself is epic. However the underlying function is not surjective. What is this a counterexample to, in terms of functors preserving epics? Which functor in particular is this about?

Whenever we’re thinking about underlying sets we are implicitly thinking about the forgetful functor of the form **Sets-with-structure** $\longrightarrow$ **Set**.

In this case we’re thinking about the functor U as shown on the right, and its particular action on the inclusion morphism shown.[†]



[†] Someone more pedantic than I am would write UN and UZ for the underlying sets of natural numbers and integers.

The point is that the inclusion $\mathbb{N} \hookrightarrow \mathbb{Z}$ as a map of monoids is epic, but its image under U (that is, where it's mapped to in **Set**) is *not* epic. Abstractly this is saying that U does not preserve epics.

This means that, in general, being an epic is not a very “stable” property, as it might not be preserved by a functor. This is where *split* epics come in.

Recall that a morphism $f: a \rightarrow b$ is a split epic if there is a morphism g making the diagram on the right commute.

$$\begin{array}{ccccc} b & \xrightarrow{g} & a & \xrightarrow{f} & b \\ & & \searrow 1_b & \nearrow & \\ & & & & \end{array}$$

The extra structure of the splitting g is enough to ensure that split epics are always preserved by functors.

Things To Think About

T 20.14 Can you show that split epics are preserved by all functors?

Suppose we have a split epic f (as in the diagram above) in a category $\mathcal{C}$, and a functor $\mathcal{C} \xrightarrow{F} \mathcal{D}$. We need to show that Ff is a split epic in $\mathcal{D}$.

Now, applying F to the original split epic diagram we certainly get a diagram in $\mathcal{D}$ as shown on the right.

$$\begin{array}{ccccc} Fb & \xrightarrow{Fg} & Fa & \xrightarrow{Ff} & Fb \\ & & \searrow F1_b & \nearrow & \\ & & & & \end{array}$$

We just have to check it commutes, which it does by functoriality.

More precisely:	$Ff \circ Fg = F(f \circ g)$	by functoriality
	$= F(1_b)$	by hypothesis
	$= 1_{Fb}$	by functoriality

One general idea about preservation is that studying whether a functor preserves certain structure might be telling us something about the functor, or it might be telling us something about the type of structure we're thinking about. Structure that is preserved by all functors is generally called *absolute*. We can think of it as particularly “stable” or strong.[†] A general rule of thumb is that anything defined by a commutative diagram will be preserved by functors, but anything involving a “for all” or “there exists” quantifier is trickier because when we move into a different category we will be quantifying over something different. For example, if we move somewhere with fewer morphisms (say because morphisms in the new place are required to preserve some structure), the quantifiers are affected in the following ways, broadly speaking:

- “For all” is now quantifying over fewer morphisms, and so it has more chance of being satisfied.
- “There exists” has fewer morphisms to choose from, so it has less chance of being satisfied.

[†] However if you're British you might be traumatized by the idea of something being “strong and stable”. (See 2017 election.)

In the next section we will pin down a bit more precisely some notions of whether a functor takes us to a place with “more” or “fewer” morphisms.

Things To Think About

T 20.15 In Chapter 16 we first looked at terminal and initial objects in **Set** and then in some categories of sets-with-structure. In some cases the terminal and initial set-with-structure had the terminal/initial set as its underlying set, and in others it didn't. Can you recall those examples and see what that is saying about some functors preserving terminal/initial objects, or not preserving them?

In **Set** any one-element set is terminal, and the empty set is initial. We can compare this with various categories of sets-with-structure.

- In the category **Pst** of posets and order-preserving maps, any one-element poset is terminal and the empty poset is initial. So the forgetful functor $\mathbf{Pst} \rightarrow \mathbf{Set}$ preserves terminal and initial objects.
- In the category **Grp** of groups and homomorphisms, the trivial group has one element (the identity) and is both terminal and initial. So the forgetful functor $\mathbf{Grp} \rightarrow \mathbf{Set}$ preserves terminal objects but not initial objects, because if we apply it to the initial object in **Grp** we do not get the initial object in **Set**.
- In the category **Top** of topological spaces and continuous maps any one-point space is terminal and the empty space is initial. So the forgetful functor $\mathbf{Top} \rightarrow \mathbf{Set}$ preserves terminal and initial objects.

Things To Think About

T 20.16 In Chapter 18 we saw that various products of sets-with-structure do have the product of sets as their underlying set. Can you express this in terms of some functors preserving products? Now consider the examples where coproducts and pushouts did not just have the coproduct/pushout of underlying sets as their underlying set. What is this saying abstractly about preservation?

We saw that products of posets, groups and topological spaces did have the product of sets as their underlying set, so the forgetful functor from any of those categories to **Set** does preserve products.

The coproduct of posets did have the disjoint union of its underlying sets as its underlying set, so the forgetful functor $\mathbf{Pst} \rightarrow \mathbf{Set}$ preserves coproducts. However the coproduct of groups was the “free product” which involved taking the disjoint union of underlying sets and then generating a whole lot of new elements for binary products involving elements from both groups. Thus the forgetful functor $\mathbf{Grp} \rightarrow \mathbf{Set}$ does not preserve coproducts; nor does it preserve pushouts, as pushouts in **Grp** are constructed by starting from the free product and then taking a quotient.

20.6 Further topics

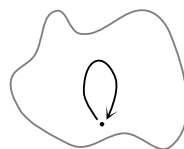
Algebraic topology

Functors between large categories of mathematical structures are sometimes how entire branches of mathematics get started. We will look at just one example here, relating algebra and topology.

Topology is the study of topological spaces, and algebraic topology is the study of topological spaces via algebra. The algebra in question is classically group theory, though more recently it has headed towards groupoid theory and higher-dimensional category theory.

The first thing one usually does in an algebraic topology course is the fundamental group. This is a way of taking a space and producing a group from it, by studying loops in the space.

First you have to choose a “basepoint” which is a point in the space where all your loops are going to start and finish. A loop is then a path in the space that starts and finishes at that point, as depicted informally here.



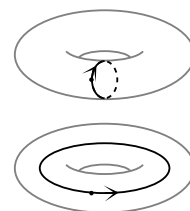
The elements of the fundamental group are the *homotopy classes* of loops. This means, essentially, that we count loops as the same if they’re not *really* different, but you can continuously deform one into the other without breaking it or going over a hole in the space.

The binary operation in the group then comes from observing that if you go round one loop and then another, you’ve got another loop — it’s just a loop where you happened to go home to the basepoint in the middle rather than waiting until the end.



This group can detect interesting structure in a space, like holes. For example the fundamental group of a circle is the integers $\mathbb{Z}$, because a loop can go round the circle any finite number of times, forwards or backwards.

The fundamental group of a torus (like the surface of a bagel) starts from two basic loops, one going “through” the hole and the other going around it, as shown on the right. Each of these loops individually generates an entire copy of $\mathbb{Z}$ as we can go round either of these any finite number of times forwards or backwards.



But we can also combine the loops, so we could go round the first one and then the second one and then maybe the first one again, for example. It turns out that the two loops commute up to homotopy, that is, it doesn’t matter which order

you do them in because one way round can be deformed into the other. So the group we get is the product $\mathbb{Z} \times \mathbb{Z}$. That was, of course, very far from a proof.

In fact the fundamental group construction is a functor $\pi_1 : \mathbf{Top}_* \rightarrow \mathbf{Grp}$. Here $\mathbf{Top}_*$ is the category of spaces-with-a-chosen-basepoint, and the morphisms are continuous maps preserving the basepoint. In the first instance the fact that π_1 is a functor means that it extends to morphisms: a continuous map between topological spaces (preserving basepoints) produces a group homomorphism between the associated fundamental groups. Functoriality tells us that identities and composition are respected. As it's a functor, we can immediately deduce that isomorphisms are mapped to isomorphisms, that is, if we have an isomorphism between topological spaces we will get an isomorphism between their fundamental groups. This is important because isomorphic topological spaces should count as “the same” and if we're going to study them via associated groups then spaces that count as the same should produce groups that count as the same.

In the previous chapter we mentioned Van Kampen's theorem, which is about when we can take a pushout of spaces and also take a pushout of their fundamental groups, and know that the results will match, enabling us to calculate fundamental groups of complicated spaces by expressing them in terms of simpler ones. If we express this in terms of the fundamental group *functor* (rather than just a “fundamental group construction”) the theorem becomes a result about when the fundamental group functor preserves pushouts.

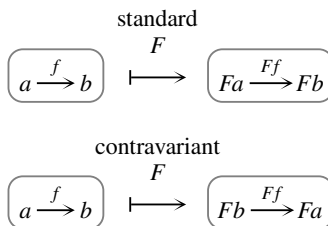
Note that looking at all this is only a starting point. For example we really need π_1 to do more than preserve isomorphisms: the correct notion of sameness for topological spaces is more subtle than isomorphism in the category $\mathbf{Top}$, as we've vaguely seen — it's the notion of homotopy equivalence, where we can continuously deform one space into another. So what we need is for homotopy equivalences to be mapped to group isomorphisms, and this is something much more subtle that basic theory about functors can't directly sort out.

Algebraic topology also uses other functors to groups, and to more complicated categories. Homology and cohomology are different ways of having a functor to groups (in fact, to Abelian groups) and to chain complexes, which are a series of Abelian groups, one for each dimension, and some “chain maps”, special group homomorphisms relating the different dimensions.

Another example of a branch of mathematics based on a functor between large categories is linear algebra, which importantly relates the category of finite-dimensional vector spaces to the category of matrices. That example is beyond our scope, but I thought I'd mention it in case it's something you've encountered and might like to look up.

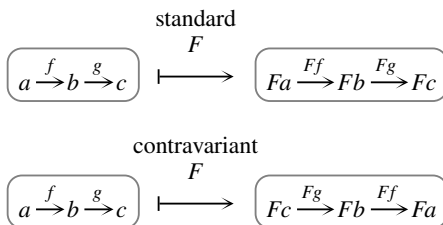
Contravariant functors

There is another type of functor I want to mention briefly, involving dual categories. It's called a *contravariant* functor, and the idea is that it reverses the direction of morphisms. The standard action on morphisms and the contravariant action are shown on the right.



To emphasize a functor that acts in the standard way, the word “covariant” is used. I’m sorry about that; it’s terribly confusing to me because it has the “co-” prefix so makes it sound like it’s reversing something (to me, anyway).

As a result of reversing the direction of the arrows, the functoriality condition for a contravariant functor also switches the order of composition, as shown on the right.



Written out in symbols this says $F(g \circ f) = Ff \circ Fg$.

As all this involves switching the direction of arrows, we can do it formally by using the dual category $\mathcal{C}^{\text{op}}$.

Definition 20.4 A *contravariant* functor is a functor $\mathcal{C}^{\text{op}} \rightarrow \mathcal{D}$.

This is a slightly hazy definition because of course every category is the opposite of another category, so every functor is formally a functor from a dual category.[†] But the spirit of contravariant functors is that they flip the “obvious” direction of the arrows. As with all dual situations I prefer never to draw arrows in $\mathcal{C}^{\text{op}}$, but instead to draw the arrows in $\mathcal{C}$ and $\mathcal{D}$ and see them flip when the functor acts.

This might sound like a contrived definition but it turns out that there are many naturally arising functors that do this rather than keep the morphisms pointing the same way, especially when we start using functors as tools for further constructions, not just as ways of comparing structure. We will see a key example in Chapter 23 when we look at the Yoneda embedding.

[†] Some people would say that a functor $\mathcal{C}^{\text{op}} \rightarrow \mathcal{D}$ is a contravariant functor $\mathcal{C} \rightarrow \mathcal{D}$ but for me that creates more confusion than it resolves.